Chapter 1: Introduction to Python

Python is a high-level, interpreted programming language known for its simplicity and readability. Created by Guido van Rossum in 1991, Python has become one of the most popular programming languages in the world. It supports multiple programming paradigms including procedural, object-oriented, and functional programming. Python's syntax is clean and easy to learn, making it ideal for beginners and experts alike.

Chapter 2: Data Types and Variables

Python supports several built-in data types including integers, floats, strings, lists, tuples, sets, and dictionaries. Variables are dynamically typed, meaning you do not need to declare their type explicitly. Lists are mutable ordered collections while tuples are immutable. Dictionaries store key-value pairs and provide fast lookups. Understanding these types is fundamental to writing effective Python code.

Chapter 3: Control Flow

Control flow in Python includes if-else statements, for loops, and while loops. The if statement evaluates a condition and executes a block of code accordingly. For loops iterate over sequences like lists and strings. While loops execute as long as a condition is true. Python also provides break, continue, and pass statements for fine-grained loop control. Indentation is used to define code blocks instead of braces.

Chapter 4: Functions and Modules

Functions in Python are defined using the def keyword. They can accept parameters and return values. Python supports default arguments, keyword arguments, and variable-length arguments. Lambda functions provide anonymous one-liner functions. Modules are files containing Python code that can be imported. The Python Standard Library provides modules for file I/O, math, datetime, and much more.

Chapter 5: Object-Oriented Programming

Python supports object-oriented programming through classes and objects. A class defines attributes and methods, while objects are instances of classes. Python supports inheritance, allowing a child class to inherit from a parent class. Encapsulation hides internal state and polymorphism allows different classes to be used interchangeably. Special methods like __init__ and __str__ define object initialization and string representation.